



ENG-39 — PostgreSQL 15 en Docker para tests de integración

Documentación del Spike de Infraestructura

VERSIÓN	1.0
FECHA	08/07/2026
SPRINT	1
ÉPICA	EP-10 (Plataforma e Infraestructura)

1. Contexto y objetivo

Hasta ahora, cualquier test que necesitara una base de datos real dependía de Supabase (remoto, compartido). Eso trae problemas:

- Los tests de integración corrían contra una base compartida, con riesgo de contaminar datos entre corridas o entre desarrolladores.
- CI no tenía forma de correr tests contra una base real sin credenciales de Supabase.
- No había manera de resetear el estado de la base entre corridas de forma barata.

Objetivo del spike: levantar PostgreSQL 15 en Docker, tanto en local como en el pipeline de CI, para que los tests de integración corran contra una base real y aislada, sin tocar Supabase.

2. Criterios de aceptación (y cómo se cumplen)

Criterio	Cómo se cumple
<code>docker-compose.yml</code> levanta PostgreSQL 15 con un comando	<code>pnpm run db:test:up</code> → <code>docker compose up -d --wait postgres-test</code>
El pipeline de GitHub Actions levanta el contenedor como <code>service</code> antes de los tests	<code>.github/workflows/integration-tests.yml</code> , bloque <code>services.postgres</code> con healthcheck
El schema de Prisma se aplica automáticamente antes de cada suite	<code>scripts/test-integration.sh</code> corre <code>prisma db push --accept-data-loss</code> antes de <code>jest</code>
Documentado en el README	Sección "Integration tests (PostgreSQL 15 via Docker)" en <code>README.md</code>

3. Cambios por archivo

3.1. Nuevos

- `docker-compose.yml` Define dos servicios:
- `postgres-test`: imagen `postgres:15`, expone `5433:5432` en el host (para no pisar un Postgres local en `5432`), healthcheck con `pg_isready`, volumen nombrado `postgres_test_data` para persistir datos entre reinicios.
- `app`: el contenedor donde corre el Dev Container de VS Code (`typescript-node:24-bookworm`), con `depends_on: postgres-test (condition: service_healthy)` para no arrancar hasta que la base esté lista. Le inyecta `DATABASE_URL` apuntando a `postgres-test:5432` (hostname de red de Compose, no `localhost`) y `SKIP_DOCKER=true`.
- `scripts/test-integration.sh` Script único que: 1. Si `SKIP_DOCKER` no es `true`, levanta `postgres-test` con `docker compose up -d --wait`. 2. Corre `prisma db push --accept-data-`

`loss` contra `DATABASE_URL` . 3. Corre `jest --config ./test/jest-integration.json --runInBand` .

Se usa tanto en local como en CI — es la misma lógica en los dos lados, solo cambia si Docker Compose participa o no (`SKIP_DOCKER`).

- `test/jest-integration.json` Config de Jest separada para integración: solo matchea `*.integration.spec.ts` , así no se mezcla con `pnpm run test` (unitarios) ni `pnpm run test:e2e` .
- `test/database.integration.spec.ts` Test de humo: instancia `PrismaClient` y corre `SELECT 1` contra la base real. Sirve para probar que la conexión y el pipeline funcionan, no testea lógica de negocio (todavía no hay tablas de dominio, ver sección 6).
- `.github/workflows/integration-tests.yml` Workflow de GitHub Actions que:
 - Se dispara en `push` / `pull_request` hacia ramas `release/**` (ver sección 6, punto sobre alcance).
 - Levanta Postgres 15 como `services:` container con healthcheck.
 - Instala dependencias con pnpm y corre `pnpm run test:integration` con `SKIP_DOCKER=true` y `DATABASE_URL` apuntando al `service` de CI.

3.2. Modificados

- `.devcontainer/devcontainer.json` Antes usaba una imagen sola (`typescript-node:24-bookworm`). Ahora:
 - Usa `dockerComposeFile: ../docker-compose.yml` + `service: app` , así el Dev Container y `postgres-test` viven en la misma red de Compose.
 - `shutdownAction: stopCompose` para bajar todo al cerrar el Dev Container.
 - `postStartCommand` corre `prisma db push --accept-data-loss` en cada arranque, para que el schema esté siempre sincronizado sin acordarse de correrlo a mano.
 - Agrega el puerto `5433` a `forwardPorts` (además del `3000` de la app).
- `.env.example` Agrega y documenta `POSTGRES_TEST_USER` , `POSTGRES_TEST_PASSWORD` , `POSTGRES_TEST_DB` , `POSTGRES_TEST_PORT` — todas con defaults razonables, solo hace falta declararlas si se quiere pisar el default.
- `package.json` Nuevos scripts:
 - `db:test:up` — levanta `postgres-test` (espera a que esté healthy).
 - `db:test:down` — la baja (conserva el volumen de datos).
 - `db:test:reset` — la baja y borra el volumen (base completamente limpia).
 - `test:integration` — corre `scripts/test-integration.sh` .
- `prisma/schema.prisma` Cambia `generator client { provider = "prisma-client" }` → `provider = "prisma-client-js"` . El provider anterior (`prisma-client` , el nuevo cliente ESM de Prisma 7) no era compatible con el import clásico `new PrismaClient()` que se usa en el test de

integración y en el resto del código. Sin este cambio, `prisma generate` no producía un cliente usable.

- `README.md` Agrega la sección "Integration tests (PostgreSQL 15 via Docker)": qué son, cómo se levantan (Dev Container vs. manual), comandos disponibles, tabla de variables de entorno y cómo funciona en CI.
- `.gitignore` Se agregó `.claude/` (config local de Claude Code de cada desarrollador, no debe versionarse). No relacionado al spike en sí, es housekeeping que surgió al revisar qué se subía en este branch.

4. Cómo usarlo

4.1. Opción recomendada: Dev Container

Al abrir el repo en el Dev Container de VS Code, todo arranca solo: `postgres-test` se levanta como sibling container, `DATABASE_URL` ya apunta ahí, y el schema se sincroniza en cada `postStartCommand`. Solo hace falta:

```
pnpm run test:integration
```

4.2. Sin Dev Container (Docker Compose manual)

Requiere Docker + Docker Compose v2 instalados en el host.

```
pnpm run db:test:up      # levanta postgres:15 en localhost:5433, espera healthcheck
pnpm run test:integration # aplica el schema (prisma db push) y corre la suite
pnpm run db:test:down    # la baja (conserva datos)
pnpm run db:test:reset    # la baja y borra el volumen
```

`test:integration` levanta la base sola si no está corriendo, así que `db:test:up` es opcional — sirve para dejarla arriba entre corridas mientras se desarrolla.

4.3. En CI

No hace falta hacer nada manual: el workflow define Postgres como `services:` container y corre `pnpm run test:integration` con `SKIP_DOCKER=true`, apuntando `DATABASE_URL` al container de CI en vez de a Compose.

5. Cómo se validó

- Se corrió `pnpm run test:integration` dentro del propio Dev Container (que ya arranca desde este `docker-compose.yml`): se vio `prisma db push` aplicar el schema y el test de conectividad pasar (`1 passed, 1 total`) contra Postgres real.

- Se confirmó por consulta directa (`information_schema.tables`) que la base conecta y responde, aunque hoy no tiene tablas (ver punto siguiente).
- Se revisó manualmente el YAML del workflow de GitHub Actions (sintaxis y triggers).

6. Limitaciones y decisiones a tener en cuenta

IMPORTANTE

Todavía no hay tablas de dominio. `prisma/schema.prisma` solo tiene `generator` y `datasource`, sin ningún `model`. Es esperado para un spike de infraestructura: el mecanismo (`db push` automático) ya funciona, pero recién va a crear tablas cuando otro ticket agregue modelos al schema. No confundir "no hay tablas" con "no funciona".

- **El workflow de CI solo se dispara en ramas `release/**`.** Hoy no corre en PRs hacia `main` ni en feature branches. Si el equipo quiere feedback de integración en cada PR (no solo antes de release), hay que ampliar los `branches:` del trigger — decisión pendiente, no tomada en este spike.
- **Puerto `5433`, no `5432`, en el host.** Elegido a propósito para no pisar un Postgres local que alguien ya tenga corriendo en `5432`. Dentro de la red de Docker Compose (Dev Container, `app` → `postgres-test`) se sigue usando el puerto interno `5432`.
- **`db push` en vez de migraciones versionadas (`prisma migrate`).** Para este spike alcanza con sincronizar el schema directo; si el equipo pasa a usar `prisma migrate` para producción, `scripts/test-integration.sh` va a necesitar el ajuste correspondiente (`migrate deploy` en vez de `db push`).

7. Próximos pasos sugeridos

- Definir y agregar los primeros `model` de dominio al schema.
- Decidir si el workflow de integración debe correr en más ramas que `release/**`.
- Evaluar migrar de `db push` a `prisma migrate` cuando haya modelos reales y se necesite versionar cambios de schema.